# CS4928 – Group RQs and Progressive Milestone Record

*Each group updates only its own row. Normally use two RQs. Enter expectations before running the corresponding work; do not rewrite earlier expectations after seeing results.*

| Group ID | Project ID | RQ1 | RQ2 | RQ3 (optional) | Bounded sample / case IDs | Method / metric or decision rule | Expected result before execution | Reproduction result (brief) | Evidence link / commit / output | Event A prediction before execution | Event A result / decision | Concrete design problem | Alternative(s) considered | Chosen intervention / no-change decision | Verification evidence | Event B prediction / expected pressure | Event B response / accepted trade-off | Final repository / tag | Final synthesis / evidence link | Main limitation / threat |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| G01 | P01 | How well can RoBERTa identify three Gang of Four patterns (Factory Method, Prototype, Singleton) in Java source code across 30 | What effect does structure-aware class and method segmentation have on design-pattern recognition |  | We will reproduce the paper's results for RoBERTa for three Gang of Four patterns: Factory | Our evaluation metrics will be Precision, Recall, and mainly F1 Score | Precision:<br>Factory Method: 0.87<br>Prototype: 0.88<br>Singleton: 0.92 | Results of 1.0 are due to small dataset size.<br>Precision:<br>Factory Method: 0.64 | REPRODUCED<br>https://github.com/BaguetteJet/CS4928-Project/releases/tag/v1.0.0 | | | | | | | | | | | |
| G02 | P02 | Across four preregistered CodelyTV Java refactoring cases, what is the quantitative difference in GenAI's mean correctness | For a selected Java refactoring case from the CodelyTV repository, how |  | The first 4 eligible Java code-smell examples from the CodelyTV/refactoring-code_smells- | Method: For the 4 CodelyTV cases, use a frozen GenAI model to diagnose the smell and generate a refactored solution. For RQ2, select | RQ1: GenAI will achieve higher mean correctness scores for GoF pattern implementation (e.g., ~8–10) than for | | | | | | | | | | | | | |
| G03 | P06 | Using a newer ChatGPT model (GPT-5.6 luna) than the original study and the authors' published prompts, on a frozen sample of 12 | When the same LLM (GPT-5.6 luna) refactors the 9 RefactoringGuru Java examples that match the |  | RQ1: 12 instances from Lie et al's dataset from the paper's replication package on GitHub | RQ1: GPT-5.6 luna, temp 0, new chat per prompt. Run the paper's P1 and P2 prompts on our 12 cases; save all answers. Success = | RQ1: P2 still beats P1. Newer model, so both higher than the paper's 1/12 → 6/12: P1 ≈ 2–4, P2 ≈ 6–9, gap ≈ 3–4 = | | | | | | | | | | | | | |
| G04 | P07 | | | | | | | | | | | | | | | | | | | |
| G05 | P08 | Reproduce vanilla LLM MoveMethod hallucination rates (H1/H2/H3) on a small sample from the paper's released corpus, comparing against | Can a simple hybrid rule reduce hallucinated recommendations without lowering Recall@1, and is a |  | 12 instance-method M | Unit of analysis: one (host clas | Reference: paper Table III (Vanilla-LLM RecallC@1 = 67% small / 44% large classes, real-world instance methods) and Table I (vanilla LLM: H1 ≈ 33%, H2 ≈ 21%). E1: LLM-only Recall@1 ≈ 5–8/12, with at least 1 H1 or H2 recommendation. E2: Hybrid has 0 H1/H2 by construction; the test is whether Hybrid Recall@1 ≥ LLM-only. E3: LLM and static top-1 agree on fewer than 6/12 cases. Reproduced: LLM-only Recall@1 ≤ 8/12 AND ≥ 1 hallucination (LLM alone unreliable). Non-reproduction: LLM-only Recall@1 > 8/12 AND 0 hallucinations. Otherwise: partial reproduction, reported as observed. | | | | | | | | | | | | | |
| G06 | P09 | Can we replicate the paper's findings, that most code refactorings only affect future changes for a short | Does the paper's rule for ideal refactoring duration still hold true on modern Java code, and |  | RQ1: Replicate 3 of the Java projects from the paper's 48 analysed project | RQ1: Collect commits and run pipeline on code<br>RQ2: Collect recent commits and run same pipeline on code | RQ1: Partial reproduction of papers findings on our sample<br>RQ2: Rule only weakly supported or not supported | | | | | | | | | | | | | |
| G07 | P10 | To what extent can the release-wise refactoring patterns reported by Noei et al. be | Are the four patters stable when the clustering is re-run under conditions |  | All 1,604 releases (157 repositories) with more than | Authors' Soft-DTW k-means, k=4, seed 9, unmodified. Reproduced if shares are within ±3 | RQ1: reproduced.<br>RQ2: shapes stable across seeds, but cluster numbers | | | | | | | | | | | | | |
| G08 | P11 | Does refactoring differ between production and test code in frequency and type? | Do a few production classes account for most test refactoring, and should the worst-affected module's |  | First 12 projects in row order of DataCollection.csv from the authors' repo : scala- | RQ1: RefactoringMiner 3.0.0 over full history to the pinned commit; file is test code if its path contains "test". Reproduced if production has 2 to 6 | RQ1.Reproduced: Production ~4.7× more refactorings than test (82.4% vs 17.6%); commits ~76% production / 11% test / | | | | | | | | | | | | | |
| G09 | P25 | | | | | | | | | | | | | | | | | | | |
| G10 | P03 | | | | | | | | | | | | | | | | | | | |
| G11 | P04 | Using the authors' published artefact and their dataset of 133 real-world God classes,<br>does ClassSplitter reproduce its reported decomposition accuracy | Does ClassSplitter's position-aware advantage transfer to Extract Class refactorings mined from Java projects outside the authors' dataset, |  | RQ1: We will study all 133 God-class refactoring cases supplied by the authors, using runner case IDs 1–133. RQ2: We will | RQ1: Run both tools on 133 published cases, compare mean MoJoFM with the reported values and check seed stability. RQ2: Compare both tools against developer splits | RQ1: We expect both overall means to agree with the saved published values within ±0.005, with ClassSplitter scoring higher than Bavota. RQ2: Our | | | | | | | | | | | | | |
| G12 | P12 | Can the supplied notebook reproduce the recorded Random Forest accuracy and Matthews correlation coefficient on the six retained comment classes, and does its implementation match the documented preprocessing and | For confirmed inline comment smells in a separate Java project, when is comment only repair sufficient, and when is code refactoring justified compared with making no change, considering |  | Use the first 50 labelled inline comments from the provided dataset to test smell detection. Then select 10 detected smells | RQ1: Run notebook as shipped (10 seeds); reproduced if within ±0.03 of 0.69 acc / 0.44 MCC. Audit code vs paper, then rerun with paper's described pipeline. RQ2: Per smell | RQ1: As-shipped reproduces; corrected pipeline does not (SMOTE appears unused in reported run). RQ2: Most smells need comment fix only; | | | | | | | | | | | | | |
| G13 | P35 | based God Class detection results across its seven classifiers be reproduced on the supplied 2,148-sample MLCQ-derived dataset? | Class predictions from the metric-based detector correspond to concrete software-design problems in |  | 3699849; 3700666; 3702984; 3705164; 3710201; 3711605; 3715636; 3717506; | IDF), both using SVM and the same 10-fold split over 2,148 samples. Evaluate accuracy, precision, recall and F1 on the 12 frozen cases, with | higher f1 score than token-based SVM on the 12 frozen cases, based off Table 2(0.6400 vs 0.4143) in the paper. The exact | | | | | | | | | | | | | |
| G14 | P05 | | | | | | | | | | | | | | | | | | | |
| G15 | P21 | When two of us independently count TP/FP/FN on Code2DFD's published outputs for four benchmark apps, how well do our | How accurately does Code2DFD recover the architecture of a new microservice app outside the |  | RQ1: Code2DFD published outputs for 4 dataset apps: piggymetrics, spring- | RQ1: 2 members label each GT item TP/FP/FN independently (components/connections/endpoints): agreement = Cohen's kappa; | Expected: kappa ≥ 0.8; pooled P≈0.96, R≈0.81, F1≈0.88 (paper data, these 4 apps), ours within ±0.05. New app: overall F1 < | | | | | | | | | | | | | |
| G16 | P22 | Can the reported difference in issue closing velocity be reproduced using the study's original sample of projects? | Does the same finding hold when applied to a new more recently created sample of projects? |  | Papers final analysed cohort: final_data_file.csv it has 58 projects that used microservice and 207 | Run papers tests on own data: Mann-Whitney and a-test then a regression that accounts for project size, team size and language. Metric: Velocity - | We expect no significant changes in how fast the two close issues like it was found in the paper. We expect the effect | | | | | | | | | | | | | |
| G17 | P23 | Can we replicate the paper's results on performance-test coverage using their provided GitHub code and data | Does adding new code metrics (or using a different machine learning model) improve how accurately we predict |  | byte-buddy,fastjson,commons-bcel,SquidLib | Ran the four original overlap and coverage scripts to compare outputs with the supplied results | Expected all four scripts to run successfully and reproduce the supplied results without changes | 1-Overlap-JMH.py: Successfully replicated exact paper metrics.2-Overlap-JUnit.py: Execution hang/timeout due to O(N^2) | https://github.com/Amilkamija/performance-test-coverage.git | | | | | | | | | | | |
| G18 | P24 | Can we reproduce the paper's finding that participants answered the security questions more accurately when they received the | How has Traffic Control changed from eShopOnContainers to its successor, eShop? | | | | | | | | | | | | | | | | | |
| G19 | P13 | Can we reproduce the detection of code smells in build scripts identified in the original study? | What are the effects of refactoring a detected build-script code smell on the build script and its associated |  | Build scripts will be collected from public GitHub repositories using key words such as "code | Analyze the collected build scripts using the frozen sniffer and using GPT4o, then compare to the results from original study. | We expect to find similar code smells as listed in the paper | | | | | | | | | | | | | |
| G20 | P14 | warning–architectural-smell relationship reproduce on our subset? | How many warnings are misleading proxies for architectural problems? |  | Of the 112 repositories the study uses we shall choose 20. These projects are ant apj argouml | We will be using the PMD PMD Source Code Analyzer to check for warnings and ARCAN to check for architectural smells | We expect to find that the correlation between the PMD warnings to have a correlation p value of 0.5 | | | | | | | | | | | | | |
| G21 | P16 | When we apply the paper's published aging-debt patterns to a bounded sample of comments from the released dataset, do we find a | In a new Java repository outside the original dataset, which self-admitted aging comments point to code that |  | RQ1: random sample of 500 comments from the released dataset (Zenodo 10.5281/zenodo.15211796 | Apply the paper's published aging-debt patterns to the sample, then classify matches as Active or Dormant using the paper's | We expect Dormant to be the large majority, as in the paper (about 96% in their gold-standard data). We count it as non- | | | | | | | | | | | | | |
| G22 | P17 | Can the reported effect of negative se | Does the effect of negative sentiment on perceived Urgency change when participants are grouped by |  | 236 Urgency ratings (vignettes #1–#4) from the 59 valid respondents, grouped by Q9 for Q1 (22 | We rerun the authors' Model 1 on Urgency with a fixed seed (swapping Q9 for Q8 in Q2), compute each group's odds ratio as P(score up) + | We expect Q1 to give Urgency ORs of about 1.95 (Agree), 1.07 (No Opinion) and 1.10 (Disagree), and Q2 to show an | | | | | | | | | | | | | |
| G23 | P18 | Can one broken-windows effect be reprodu | Does a similar debt-amplification pattern appear in a fixed AI-a |  | All submissions in data.csv from BWTSE/Data v1.4 (paper: 51 from 29 participants;), | We refit the authors' Model 1 (validation-logic reuse ~ high_debt_version + programming_experience + (1 \| | We expect β ≈ −1.8 (CrI ≈ [−3.1, −0.5]) as in Table 2, Model 1, meaning reuse of validation logic is more likely in the low-debt | | | | | | | | | | | | | |
| G24 | P19 | When analysing a bounded sample of file structure outliers from the authors released dataset can we reproduce the main categories and | Do the main causes of the file structure outliers identified in the original study also occur in a bounded sample of | To what extent do file structure outliers associated with design problems | OLD–Unit of Analysis: File-structure changes Source: GitHub repositories | RQ1: For each of the 15 Java commits in the bounded sample, compare the maximum depth of tracked files at the commit with its | RQ1: We expect our depth increases and outlier decisions to match the authors results for at least 13 of the 15 commits | | | | | | | | | | | | | |
| G25 | P20 | Can we reproduce the paper's model ranking (M2 > M1 > M0) and its ownership finding using the replication package? | Does a team's commit share in a repo predict clone introduction as well as the original M2 model? | | | | | | | | | | | | | | | | | |
| G26 | P26 | Can PR-level change-impact ranking be reproduced on the selected project? | Which high-risk predictions correspond to real review/defect evidence? |  | Unit: one pull request.<br>Source: github.com/arduino/Arduino, branch master, commit a0df6e0e83b652c72bc78b0a1376c54d6ebc3bee.<br>Sample: first 10 eligible PR's in ascending PR number created on or after 01/01/2019. Eligible = merged or open, base master, modifies >= 1 existing .java file (paper §4.2.1–4.2.2). PR ID's: #8349, #8429, #8458 | RQ1: Per PR: changed .java files → changed and impacted methods (artefact callgraph-server) → 5 metrics with the paper's thresholds (Tables 3–7) → risk score with default weights (Fig. 8) → rank. REPRODUCED = ≥8/10 PRs scored, >1 risk category. NOT REPRODUCED = valid run, <8 PRs scored. INCONCLUSIVE/BLOCKED = no valid run.<br>RQ2: High-risk prediction = top 3 PRs by risk score. Evidence = bug label or a later bug-fix PR touching the same files. List which high-risk | All metrics and a risk score can be computed for every PR, giving a ranking that separates the PRs. The top 3 (high-risk) PRs will show more review/defect evidence than the bottom 3 | | | | | | | | | | | | | |
| G27 | P28 | To what result can the information diffusion range reported by the paper be repeated using the authors algorithm and Trivago's code review | How does Retrieval Augmented memory affect information diffusion coverage compared to the paper's |  | RQ1: We propose selecting the Trivago participants (364) dataset. The original network | RQ1: Run original information diffusion algorithm from author to calculate range. Compare to corresponding source level | RQ1: 85% information Diffusion Coverage (Maximum) RQ2: RAG system should perform atleast equal or slightly | | | | | | | | | | | | | |
| G28 | P15 | | | | | | | | | | | | | | | | | | | |
| G29 | P27 | | | | | | | | | | | | | | | | | | | |
| G30 | P29 | If we run the authors' own code and data on the project-level test split, do we get the same number of records removed at each preprocessing | Does the preprocessing pipeline behave the same way on new data, and is restructuring its duplicated |  | Model: T5-small. RQ1: first 12 records of the authors' project-level test split. RQ2a: first 12 records of | Method: run the authors' scripts unchanged (T5-small); rerun on new data; compare refactoring vs leaving the duplicated code | RQ1: counts match Table 4; T5 only weakly affected by preprocessing. RQ2a: similar removal rates on new data | | | | | | | | | | | | | |
| G31 | P30 | To what extent can the PatchTrack classification results (the distribution of Patch Applied / Patch Not Applied / No Patch Exists PRs, and the | Can PatchTrack be refactored/modernised and extended to study whether disclosure of AI use affects |  | N selected pull requests from the study's released dataset containing self-admitted ChatGPT usage | Examine each selected pull request.<br>Examine the review discussion/comments | Paper findings should approximately match paper findings | | | | | | | | | | | | | |
| G32 | P31 | What is the prevalence of "Assertion Roulette" (AR) and "Lazy Test" (LT) smells in GPT-3.5 generated unit tests versus EvoSuite-generated tests for the Defects4J benchmark, as detected by TsDetect? | How does applying an AI-assisted refactoring pattern (extracting explicit assertions and modularizing test methods) resolve AR and LT in a bounded sample, and what is the trade-off in test |  | WEEK 6 · LAB 4 - PROJECT STUDIO | 1. Reproduction: Run the provided TsDetect pipeline script on the Defects4J JSONL datasets. Compute prevalence percentages for AR and LT. 2. Refactoring (Decision): Select 5-10 test classes heavily flagged with AR and LT. Apply | Expected result: Consistent with the paper, we expect both GPT-3.5 and EvoSuite tests to show high prevalence of AR and LT. The refactoring intervention should eliminate these structural smells but may increase | | | | | | | | | | | | | |
| G33 | P32 | To what extent can the paper's Phase 2 comparison between treatment (developers evolving AI- | Do two other maintainability measures in the package, PMD rule violations and | Can the duplicated, hard-to-re-run analysis code in the | Execute the Reproduction | Two-sided tests at α = 0.05, no multiple-testing correction, re-run from the authors' published data and | RQ1: No significant difference between treatment and control on any of the four Phase 2 | | | | | | | | | | | | | |
| G34 | P33 | On a fixed sample of BUMP breaking updates, does Byam with DeepSeek V3 reproduce the paper's reported build success rate (BSR) and error fix rate (CEFR) for the baseline prompt (P1) and the full-context prompt (P8)? | Can Byam be refactored to work with a newer model, such as Claude Opus, to improve its error fix rate for baseline prompt (P1) and the full-context prompt (P8)? | with current-generation models, such as Claude Opus, does P8 still beat P1, or does the benefit of added context shrink as models get stronger? | Unit: One eligible Java breaking-dependency update case Source: The BUMP breaking dependency update cases provided through the artists replication artefact. The | For each selected case, reproduce the breaking dependency update and capture the resulting compilation failure. Construct a fixed LLM prompt using the pre-declared prompt configuration and provide the relevant client code and | We expect the LLM to successfully repair some, but not all, breaking dependency updates. Based on the original study, richer contextual information such as the erroneous line and API differences is | | | | | | | | | | | | | |
| G35 | P34 | Can the bounded JDK/build-tool success pattern be reproduced? | How effective is a simple JDK-switch repair before code changes? |  | First 20 eligible projects from results.csv, sorted alphabetically by project name: 0pandadev/nextron, 1135/equationexploit, 123yyh123/xiaofanshu, 15knots/cmake4eclipse, 1991wangliang/lorne_core. | Build each project using JDK 11, 17, 21 and 23; record build success/failure and compare success patterns. For JDK 23 failures, test whether switching to JDK 21, 17 or 11 succeeds without code changes. | Build success will vary across JDK versions, broadly following the original study. Some JDK 23 failures are expected to build successfully using an earlier JDK without code changes. | | | | | | | | | | | | | |

G

The point of this studio

Every group is working on a different study, so there is no common repository, dataset or "correct answer" today. The common task is the process: follow the frozen M1 plan, execute the bounded reproduction, preserve raw evidence, handle deviations transparently, and reach a defensible current reproduction status

1 · Re-open the frozen M1 contract

Before running anything substantial, read the M1 entry your group froze in Week 5 alongside the assigned study/project pack. Do not replace the RQ, sample, metric or decision rule because you now think a different route may give a better-looking result

M1 item Confirm before execution

RQ + target The exact assigned RQ(s) and original result/claim you are reproducing

Bounded sample The frozen unit, source/version, size/range, selection rule and exclusions

Method The frozen inputs → procedure → output route, including material parameters/configuration

Metric / result What output you will calculate or observe and how it connects to the RQ

Decision rule The pre-declared rule for interpreting REPRODUCED / NOT REPRODUCED / INCONCLUSIVE

Held-out boundary Only the initial material currently released to your group. Do not use Event/held-out material

No answer-chasing

A reproduction is useful even when it does not match the paper. Your job is to produce trustworthy evidence, not to keep changing the experiment until the original result appears

2 · Prepare one traceable run

Set up the exact artefact/data/version required by M1 and make the execution route explicit. In your project repository or evidence location, preserve enough information for another person to understand what was run without relying on your memory

Preserve Minimum evidence

Input identity Dataset/repository/release/artefact version and the bounded items actually used

Environment Relevant tool/language/library/version or configuration that materially affects the run

Execution route The command, script, notebook step or other procedure used to generate the result

Raw evidence Logs, CSV/JSON/text output, result files or other original machine/artefact output. Prefer raw evidence over screenshots where possible

Derived result The table, count, metric, comparison or other output calculated from that raw evidence

3 · Prove the pipeline on a small valid case

Before launching the whole bounded sample, execute the method on one small representative item or the smallest valid portion of the frozen sample. This is a pipeline check: can the intended input travel through the intended procedure and produce the intended form of output? It is not permission to choose only the easiest case for the final result

If the pilot exposes a problem

CS4928 | Week 6 · Lab 4 · Project Studio · Reproduction

Fix ordinary execution faults where possible. If the fix changes a material M1 decision — sample, method, parameter, metric or interpretation rule — preserve the original M1 entry and record a deviation BEFORE the new run: what changed, why it was necessary, and how it may affect interpretation

4 · Run the bounded reproduction

Now execute the frozen sample. Keep failed/awkward cases visible. Do not silently remove an item, change a threshold, tune a parameter or switch to a different metric because the current output is inconvenient

Allowed Not allowed

Repair a path, dependency or command error that does not

change the empirical method

Change the frozen sample after seeing which items give favourable

results

Repeat a run when the first execution is demonstrably

invalid; preserve why it was invalid

Drop failed cases without applying the frozen exclusion rule and

recording them

Make a necessary methodological deviation and record it

before rerunning

Change the metric, tolerance, parameter or decision rule after seeing

the output without declaring a deviation

Use GenAI to troubleshoot code/environment, then verify the

proposed fix yourself

Treat an AI explanation or generated number as empirical evidence

Preserve each valid raw output used in the analysis. Overwrite the only copy of raw evidence with a cleaned/edited result

Stop and record a genuine blocker when a valid run cannot

be produced

Use held-out/Event material to rescue or enlarge the Week 6 result

5 · Build the evidence chain before interpreting

Your result must be traceable backwards. Pick at least one result you plan to report and make sure you can point from the claim all the way back to the raw execution evidence that produced it

FROZEN

INPUT

RAW EXECUTION

EVIDENCE

DERIVED RESULT / COMPARISON

The

bounded

cases/vers

ion

selected

by the M1

rule

The actual

logs/files/outputs produced

by the reproduction

procedure

The metric/table/ordering/count used to compare with the original study

6 · Compare with the original study

Only after you have a valid result should you apply the M1 decision rule. A different result is not automatically a mistake; a matching result is not automatically trustworthy. First decide whether the run itself is valid and sufficiently complete

Current status Use it when

REPRODUCED A valid completed bounded run satisfies the pre-declared criterion for the original result/claim

NOT REPRODUCED A valid completed bounded run fails the pre-declared reproduction criterion

INCONCLUSIVE /

BLOCKED

Execution, data, environment or evidence prevents a valid comparison

IN PROGRESS The pipeline is valid but the bounded sample/result is not yet complete. This is a progress state, not a scientific outcome; complete it before M2

7 · Coordinator reproduction checkpoint

Show the coordinator evidence from your own study. The coordinator does not need every group to use the same tool or output format; they need to see that your evidence chain is real, bounded and consistent with M1

CS4928 | Week 6 · Lab 4 · Project Studio · Reproduction

Show What should be visible

Frozen M1 The RQ, bounded sample, method/metric and decision rule you committed to before the run

Executable route The actual command/script/notebook/procedure used for this reproduction